

Lab 4: VibeCoding

Conversation Transcript and Python Game Fixes

Project: Skyscraper Stack (Pygame)

Environment: VS Code, Windows PowerShell, Python 3.14.3, pygame-ce 2.5.8

This document captures the key conversation steps, troubleshooting commands, README requirements, identified bugs, and the corrected `game_engine.py` supplied during the session. Screenshots are represented by their described terminal results rather than embedded as full-resolution images.

1. Assignment overview

The assignment requires cloning or forking the assigned repository, reading its README, running the Python game or puzzle, recording a 10-second video before changes, using Vibe Coding prompts to fix bugs and add README features, recording a 10-second video after changes, and pushing changes to the student's own repository. Separate commits are required for each task. Do not raise a pull request to the main SETAPESU26 repository.

Final submission requirements shown in the project's README:

- A 10-second gameplay video before changes, showing the bug or broken behavior.
- A 10-second gameplay video after changes, showing the bug fixed and new features working.
- The Chat/LLM conversation link with the chat history.

2. Environment setup and pygame installation

The user was working in:

C:\Users\Sumukh Hegde\Downloads\54_skyscrapper_stack-main

Initial command: `pip install pygame`

The install failed with `ModuleNotFoundError: No module named 'setuptools._distutils.msvccompiler'`, followed by an error building pygame.

Environment checks showed:

- Python 3.14.3
- pip 26.2.1
- pygame was not installed

Installing pygame with `--only-binary=:all:` returned "No matching distribution found for pygame" under the initial package selection. Rather than installing another Python version immediately, the user tried `pygame-ce`.

```
python -m pip install --upgrade pip
python -m pip install --only-binary=:all: pygame
python -m pip install pygame-ce
python -c "import pygame; print(pygame.version.ver)"
```

The `pygame-ce` installation succeeded. The reported version was `pygame-ce 2.5.8 (SDL 2.32.10, Python 3.14.3)`.

3. Launching the game and identifying the missing file

```
python main.py
```

An initial run showed `FileNotFoundError` for 'portfolio.csv'. The guidance was to check whether the CSV existed in the project and confirm that VS Code was running the intended copy of `main.py`. The user later indicated the game ran, so the conversation moved on to inspecting `README.md` and the game source files.

4. README requirements

The README output displayed a Submission Checklist with three deliverables:

1. A 10-second video of gameplay before changes, showing the bug/broken behavior.
2. A 10-second video of gameplay after changes, showing the bug fixed and new features working.
3. The Chat/LLM used page link, with the chat history.

The user was advised to record the original gameplay before changing code.

5. Existing source code and identified bugs

The user provided `game/block.py` and `game/game_engine.py`.

Identified issues in `GameEngine.drop_block()`:

- The original code assigned `is_successful_drop = overlap <= 0`, which incorrectly treated a complete miss as a successful drop.
- It then used `max(10.0, overlap)`, which could create a fake block despite no real overlap.
- Correct behavior: if `overlap <= 0`, set `game_over = True` and return without adding a block or increasing score. If `overlap > 0`, create a stationary block using the exact intersection from left to right.

Restart mismatch:

- The on-screen message said "Press [Space] or [R] to Play Again".
- The original event handler restarted on R or mouse click, but not Space.
- The fix supports Space, R, and left mouse click after game over.

Boundary movement:

- `Block.update()` should clamp the moving block to the 20-pixel side margins and reverse its direction at either edge.

6. Vibe Coding prompt used

I am working on a Python Pygame game called Skyscraper Stack. Inspect `game/block.py` and `game/game_engine.py`.

Fix the bugs in the existing code while preserving its architecture and visual style.

1. Fix `GameEngine.drop_block()`: a drop is successful only when the moving block has a positive horizontal overlap with the top stacked block. If there is no overlap, set `game_over = True` and do not add a block or increase the score. If there is overlap, create a stationary block using the exact overlapping horizontal interval and width.
2. Fix the restart controls so the game behavior matches the on-screen instructions. Support restarting with Space or R after game over, and retain mouse-click restart if practical.
3. Check the block boundary movement in `Block.update()` so moving blocks stay within the horizontal screen boundaries.
4. Preserve the existing game design, scoring convention, and file structure. Do not add unrelated features or rewrite the entire project.
5. Explain each change and provide manual test steps for a successful drop, a complete miss on either side, and restarting after game over.

Do not claim the game has been tested unless it has actually been run.

7. Corrected game/game_engine.py

Replace the existing game/game_engine.py with the following code. It is based on the code provided in the conversation. It has not been run against the full project in this session, so run the manual checks below before committing.

```
import random
import pygame
from game.block import Block

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.block_height = 28
        self.base_width = 180

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 28)
        self.font_big = pygame.font.SysFont(None, 46)

        self.reset()

    def get_color(self, index):
        palette = [
            (230, 75, 75),
            (240, 140, 45),
            (245, 210, 50),
            (60, 195, 110),
            (50, 150, 240),
            (165, 80, 225),
        ]
        return palette[index % len(palette)]

    def reset(self):
        self.score = 0
        self.game_over = False

        base_x = (self.width - self.base_width) // 2
        base_y = self.height - 60

        base_block = Block(
            base_x, base_y,
            self.base_width, self.block_height,
            self.get_color(0), speed=0
        )

        self.stack = [base_block]
        self.spawn_active_block()

    def spawn_active_block(self):
        top_block = self.stack[-1]
        next_y = top_block.y - self.block_height - 4
        speed = min(10.0, 4.5 + len(self.stack) * 0.35)
        color = self.get_color(len(self.stack))

        start_x = (
            25 if random.choice([True, False])
            else self.width - 25 - top_block.width
        )

        self.active_block = Block(
            start_x, next_y,
            top_block.width, self.block_height,
            color, speed=speed
        )

    def drop_block(self):
        if self.game_over:
            return

        top_block = self.stack[-1]
        active = self.active_block

        # Calculate the horizontal intersection.
```

```

left = max(active.x, top_block.x)
right = min(
    active.x + active.width,
    top_block.x + top_block.width
)
overlap = right - left

# A zero-width or negative intersection is a miss.
if overlap <= 0:
    self.game_over = True
    return

# Keep only the overlapping part of the block.
new_block = Block(
    left,
    active.y,
    overlap,
    self.block_height,
    active.color,
    speed=0
)

self.stack.append(new_block)
self.score += 1

# Shift the tower down when it grows too high.
if new_block.y < 180:
    shift_amount = self.block_height + 4
    for block in self.stack:
        block.y += shift_amount

self.spawn_active_block()

def handle_event(self, event):
    if self.game_over:
        if (
            event.type == pygame.KEYDOWN
            and event.key in (pygame.K_SPACE, pygame.K_r)
        ):
            self.reset()
        elif (
            event.type == pygame.MOUSEBUTTONDOWN
            and event.button == 1
        ):
            self.reset()

    return

if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
    self.drop_block()
elif (
    event.type == pygame.MOUSEBUTTONDOWN
    and event.button == 1
):
    self.drop_block()

def update(self):
    if not self.game_over:
        self.active_block.update(self.width)

def render(self, screen):
    screen.fill((24, 27, 36))

    title_surf = self.font_title.render(
        "Skyscraper Stack", True, (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (self.width // 2 - title_surf.get_width() // 2, 16)
    )

    score_surf = self.font_hud.render(
        f"Height: {self.score}", True, (255, 220, 80)
    )
    screen.blit(
        score_surf,

```

```

        (self.width // 2 - score_surf.get_width() // 2, 54)
    )

    for block in self.stack:
        block.render(screen)

    if not self.game_over:
        self.active_block.render(screen)

    if self.game_over:
        overlay = pygame.Surface(
            (self.width, self.height), pygame.SRCALPHA
        )
        overlay.fill((0, 0, 0, 195))
        screen.blit(overlay, (0, 0))

        over_surf = self.font_big.render(
            "TOWER COLLAPSED!", True, (240, 75, 75)
        )
        screen.blit(
            over_surf,
            (
                self.width // 2 - over_surf.get_width() // 2,
                self.height // 2 - 40
            )
        )

        final_surf = self.font_hud.render(
            f"Final Height: {self.score}",
            True,
            (255, 255, 255)
        )
        screen.blit(
            final_surf,
            (
                self.width // 2 - final_surf.get_width() // 2,
                self.height // 2 + 10
            )
        )

        restart_surf = self.font_hud.render(
            "Press [Space] or [R] to Play Again",
            True,
            (200, 200, 200)
        )
        screen.blit(
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                self.height // 2 + 50
            )
        )

```

8. Manual test plan

1. Run the game from the project root:

```
python main.py
```

2. Successful drop:

Wait until the moving block overlaps the tower and press Space. The game should add only the overlapping width and increase the score by one.

3. Complete miss:

Allow the moving block to pass fully to either side, then press Space. The game should enter game-over state. The score should not increase for the missed drop, and no block should be added.

4. Restart:

After game over, press Space. The game should reset. Repeat and test R. A left mouse click should also restart.

5. Boundary movement:

Observe that moving blocks reverse direction at the left and right margins rather than moving off-screen.

6. Regression check:

Confirm the title, colors, score display, tower shift, and regular gameplay still behave as expected.

9. Git workflow for separate commits

After testing the bug fix, inspect the changed files:

```
git status
```

Stage and commit the bug-fix files:

```
git add game/block.py game/game_engine.py
```

```
git commit -m "Fix block overlap and restart controls"
```

Only stage files that were actually changed and verified. Make a separate commit for each additional feature implemented. Push to the student's personal repository, not the main SETAPESU26 repository. Do not open a pull request to the main repository.

10. Final submission checklist

☐ 10-second before video, showing original broken behavior

☐ 10-second after video, showing the fixes/features working

☐ Updated Python source code

☐ Chat/LLM conversation link and exported chat history

☐ Separate commits for bug fixes and feature tasks

☐ Code pushed to personal GitHub repository

☐ No pull request to SETAPESU26 main repository

Note: The transcript is a structured record of the key messages and technical content from this conversation, not a raw platform export. To submit the official conversation link, use the ChatGPT share/export controls available in your account.